
Sample-Efficiency of DeepRL Using Kolmogorov–Arnold Networks

Anonymous Author(s)

Affiliation

Address

email

Abstract

Deep reinforcement learning has achieved substantial performances over classical control approaches. Yet, a central challenge to learning in real-world applications is acquiring costly samples. Kolmogorov-Arnold Networks are a recently proposed architecture, that can particularly well learn physical relationships for control problems, with significantly higher parameter efficiency and interpretability when compared with Multi-Layer-Perceptron architectures. In this work, we study sample-efficiency using computational experiments, covering the Feynman dataset, the Gymnasium RL benchmark. The results show, that similar performance can be achieved with 40% less samples using the Kolmogorov-Arnold architecture, and that relative performance improvements up to 50% can be achieved during the training process. The observed gains are robust to varying levels of noise in rewards. These results highlight the potential of the Kolmogorov-Arnold architectures for more sample-efficient reinforcement learning.

1 Introduction

Reinforcement learning (RL) has become a powerful paradigm for learning feedback controllers directly from interaction with a system [1]. Classical control methods, including model-based and model predictive control, can be highly effective when accurate dynamics models and tractable objectives are available, but these assumptions are often difficult to satisfy in complex, nonlinear, or noisy environments [2]. Value-based RL methods such as Q-learning instead learn an action-value function that estimates the long-term return of taking an action in a given state [3]. Deep RL extends this idea by replacing tabular value functions or policies with neural networks, enabling controllers to approximate high-dimensional and nonlinear state-action relationships [4, 5]. This flexibility has made neural RL attractive for control tasks in robotics, networking, and other cyber-physical systems where the controller must cope with sensor noise, nonlinear dynamics, delayed effects, and changing operating conditions.

A central challenge for RL in control is the need to balance exploration and exploitation while learning from limited interactions [6]. Additional challenges include designing reward functions that encode the intended behavior and maintaining stability while the data distribution changes with the policy [7]. For safety-critical control, an additional limitation is that standard neural policies are difficult to inspect, verify, or translate into controller equations [8]. These issues motivate function approximators that can learn strong policies from fewer interactions while retaining enough structure to support analysis and interpretation [9]. The most central practical challenge is that learning a useful controller can require a large number of environment interactions [10]. Each sample in a control problem is not merely a labeled example, but a transition generated by running a policy in a simulator, a digital twin, or a physical system [11]. When running an RL environment is computationally expensive, slow, unsafe, or subject to hardware wear, sample inefficiency becomes a practical barrier to deployment.

38 Kolmogorov-Arnold Networks (KANs) are a recently proposed class of neural network architectures
 39 inspired by the Kolmogorov-Arnold representation theorem, which states that any continuous mul-
 40 tivariate function can be expressed as a finite superposition of univariate functions and summation
 41 operators [12, 13]. Unlike traditional neural networks such as multilayer perceptrons (MLPs), which
 42 rely on linear transformations followed by fixed nonlinear activation functions, KANs replace scalar
 43 weights with learnable univariate functions defined along the edges of the network [14]. This for-
 44 mulation allows the model to directly learn functional relationships between variables rather than
 45 approximating them only through weighted sums of fixed activation functions. As a result, KANs
 46 offer improved interpretability and significantly higher parameter efficiency while retaining strong
 47 approximation capabilities. Recently, KANs have gained considerable attention, particularly in
 48 areas where understanding the underlying functional structure of the data is important [15, 16, 17].
 49 KANs are therefore particularly well suited for applications such as symbolic regression, scientific
 50 discovery, and physics-informed modeling, where both data efficiency and model transparency are
 51 essential [15, 16]. These properties have also motivated the use of KANs in RL-based control,
 52 including PPO-based continuous control, robotic policy learning, load balancing, and explainable RL
 53 [18, 19, 20, 21, 22].

54 Existing evidence on the efficiency of KANs is promising. Prior work reports that smaller KANs
 55 can match or outperform larger MLPs in several data-fitting and scientific-computing examples,
 56 suggesting favorable neural scaling behavior and parameter efficiency [14]. In structured settings,
 57 KANs have been shown to reduce the need for expensive high-fidelity data and to perform well
 58 in low-data industrial fault detection [23, 24, 25]. Theoretical work also gives sample-complexity
 59 guarantees for residual KANs under smoothness and noiseless-sample assumptions [26]. Yet, in more
 60 systematic low-data comparisons, MLPs with individualized learnable activations can outperform
 61 KANs when only a few hundred samples are available [27]. These mixed findings, together with the
 62 absence of a systematic assessment of sample efficiency in RL-based control applications, motivate
 63 this work: it remains unclear when KANs improve sample efficiency over MLPs under comparable
 64 experimental budgets.

65 In this work, we study the sample-efficiency capabilities of KANs relative to MLPs in RL-based
 66 control.

67 2 Related Works

68 **Kolmogorov-Arnold networks and learning guarantees.** KANs were introduced as an alternative
 69 to MLPs in which learnable univariate functions are placed on edges rather than fixed nonlinearities
 70 on nodes [14]. This architecture has motivated rapid follow-up work because it connects neural
 71 approximation to interpretable function decompositions and can use fewer parameters in some
 72 scientific-computing and symbolic-regression settings [15, 16]. Theoretical work has begun to
 73 formalize these intuitions by proving approximation and sample-complexity guarantees for residual
 74 KANs on smooth function classes (including Besov spaces) [26]. These guarantees are important,
 75 but they are typically stated under assumptions that differ from RL control, where targets are noisy,
 76 bootstrapped, and induced by a changing policy.

77 **Low-data and fixed-sample empirical comparisons.** Empirical evidence for KAN sample efficiency
 78 remains mixed. The original KAN experiments show favorable scaling and strong performance with
 79 compact models, but they do not isolate environment-sample efficiency under fixed compute and
 80 fixed interaction budgets [14]. Follow-up works directly study low-data regimes and find that MLPs
 81 with individualized learnable activations can outperform KANs with substantially fewer parameters,
 82 especially when only a few hundred samples are available [27]. Further studies show that KANs
 83 and MLPs trade advantages across regular, irregular, discontinuous, and noisy functions, with KANs
 84 converging quickly in some cases but failing to dominate across stabilized test loss [28]. These
 85 comparisons suggest KANs should not be treated as universally more sample-efficient than MLPs.

86 **Structured and expensive-data settings.** More positive results appear when the problem structure
 87 matches the inductive bias of KAN edge functions. Some works propose multifidelity KANs that
 88 combine abundant low-fidelity data with limited high-fidelity data, reducing the amount of expensive
 89 high-fidelity data needed in scientific machine learning [23]. Furthermore, studies evaluate KAN
 90 autoencoders for fault detection across varying training-set sizes and find that several KAN variants
 91 match or surpass an MLP-based orthogonal autoencoder with substantially less data [24]. These

92 results are highly relevant for control domains with costly simulation or measurement, but they are
93 supervised or unsupervised learning studies rather than online RL studies.

94 **KANs in RL and control.** Several recent works integrate KANs into RL pipelines. Prior works
95 replace MLP actor and critic components in PPO with KANs and report comparable performance with
96 fewer parameters on DeepMind Control Proprio robotics benchmarks [18]. Similarly, studies propose
97 KAN-PPO and MultKAN-PPO for continuous control, finding robust performance across several
98 configurations while noting sensitivity to dropout design [19]. Other studies focus on KAN-based
99 PPO agents for CartPole and radio-access-network control, showing that KAN agents can match
100 MLP performance with far fewer parameters and can be converted into sparse symbolic policies [22].
101 Moreover, works propose a KAN actor in PPO for load balancing and emphasize the ability to extract
102 interpretable controller equations [21]. Together, these works support the feasibility of KAN-based
103 RL, but they primarily emphasize parameter efficiency and interpretability rather than controlled
104 sample-efficiency comparisons.

105 **KANs for robotic policies and world models.** KANs have also been used in policy-learning systems
106 outside the standard actor-critic replacement setting. KAN Policy can be used for diffusion-based
107 visuomotor policy learning, and works doing so report smoother trajectories, shorter execution times,
108 and improved success rates relative to diffusion-policy baselines [20]. *QuadKAN* for vision-guided
109 quadruped locomotion reports faster convergence, improved returns, lower action jitter, and better
110 robustness in structured gait-control settings [25]. *KAN-Dreamer* [29] replaces selected components
111 with KAN or FastKAN [30] modules, finding that FastKAN is promising for reward and continue
112 prediction but that direct replacement of visual encoders or actor-critic modules can harm sample
113 efficiency. This component-level evidence reinforces the need to distinguish where KANs are useful
114 in a RL stack rather than assuming that every MLP can be replaced beneficially.

115 3 Experiments

116 3.1 Feynman Sample-Efficiency Benchmark

117 The first experiment tests the core approximation question in a controlled supervised setting, before
118 moving to full RL control. If KANs are more sample-efficient because their edge functions provide a
119 useful inductive bias, this advantage should already be visible when learning smooth physical laws
120 from few samples. For this purpose, we follow Liu et al. and use the Feynman benchmark, which
121 contains closed-form physics equations derived from the Feynman Lectures and related symbolic-
122 regression benchmarks [14, 31, 32]. Each benchmark task is a regression problem, where each
123 Feynman Lecture equation is independently sampled to generate training sets (varying size) and test
124 sets (fixed size). The primary evaluation metric is root mean squared error (RMSE).

125 3.2 RL Gymnasium Benchmark

126 The second experiment studies the sample efficiency of KANs and MLPs in the context of DeepRL
127 for control. For this purpose, we use the RL benchmark "*Gymnasium*" [33], which contains a
128 multitude of different physics environment to study RL. We chose the subset of "Classic Control"
129 as state variables are used for learning, rather than image or video streams, as KANs are known to
130 be particularly suitable to learn physical relationships. Furthermore, we chose the proximal policy
131 optimization (PPO) learning strategy, as it is a standard, robust policy-gradient method for continuous
132 and discrete control problems [34]. Following environments are included in the benchmark: "Acrobot-
133 v1", "CartPole-v1", "MountainCar-v0", "MountainCarContinuous-v0", and "Pendulum-v1". Each
134 benchmark task provides a comparable learning curve with best rewards achievable (convergence
135 limits) enabling a comparison.

136 3.3 RL Gymnasium Benchmark With Noised Rewards

137 The third experiment studies the robustness of the sample efficiency gains of KANs over MLPs in
138 the context of DeepRL in control applications, for varying reward noise levels (Gaussian noise with
139 increasing magnitudes).

3.4 Comparison and Sample-Efficiency Assessment

Different model complexities and training parameters were explored via grid-search to identify optimal performance. KAN and MLP models were matched by model complexity measured in number of parameters to ensure a fair comparison. Number of training samples (e.g., Feynman samples, or environment episodes) to achieve a certain performance were used to assess the enhanced sample-efficiency of KANs over MLPs. Specifically, we compare ReLU and Sigmoid for MLP, and B-Splines and radial base functions (RBF) for KAN networks, to cover a multitude of different function space shapes.

4 Results

The Feynman benchmark shows a clear low-sample crossover in favour of KANs after 50 training samples, as shown in Figure 1. At the matched parameter scale used for Figure 1, the best KAN family reduces mean NRMSE from 0.200 to 0.163 at 50 samples and from 0.127 to 0.060 at 1000 samples. This corresponds to a relative NRMSE improvement that rises from 13.9% at 50 samples to 49.3% at 1000 samples. Equivalently, KANs reach several MLP error levels with substantially fewer training samples once the target error is below the early-data regime.

The Gymnasium experiment reproduces the same pattern in control, although the size of the effect depends strongly on the environment. Across Acrobot, CartPole, MountainCar, and Pendulum, the mean best-KAN advantage peaks at about 41.6% after 2250 episodes and then narrows to about 7.5% by the final checkpoint. Individual environments show larger peak gains, including 50.7% on Acrobot, 46.1% on CartPole, 42.2% on MountainCar, and 85.3% on Pendulum. The sample-reduction curves in Figure 2 show that the practical benefit is concentrated before the MLP reaches its terminal return. After many training episodes, both architectures approach similar return levels, so the main advantage of KANs is faster learning rather than a consistently higher asymptote.

The noisy-reward benchmark indicates that these gains are not an artefact of clean reward feedback. Adding Gaussian reward noise increases variance, but the KAN and MLP comparison keeps the same qualitative shape across the tested noise scales in Figure 5. KANs therefore appear most useful as an early-learning architecture rather than as a way to improve final return after large interaction budgets.

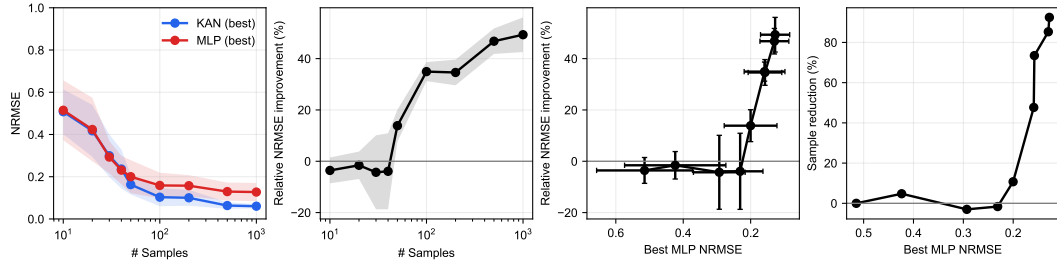


Figure 1: Experiment 1: Sample Efficiency in Feynman Benchmark.

5 Discussion

Overall, the experiments support the claim that KAN actor-critic models can improve sample efficiency when the task contains smooth low-dimensional structure. The strongest gains occur in the early and middle part of training, which is the regime that matters most when simulator calls, measurements, or hardware trials are expensive. The effect is not uniform across tasks, and the final returns are often close once both model families receive many samples. This suggests that KANs should not be treated as a universal replacement for MLPs, but as a useful inductive bias for low-data control problems. A practical route for deployment is to train with KANs during the scarce-sample phase and then distil the learned policy into a cheaper MLP when inference cost is the bottleneck. The present study is limited to the Feynman benchmark and low-dimensional Gymnasium Classic Control tasks, so larger continuous-control, robotic, and safety-critical benchmarks remain necessary. Future work should separate actor-only and critic-only effects, test stricter parameter matching, and measure wall-clock cost alongside environment-sample efficiency.

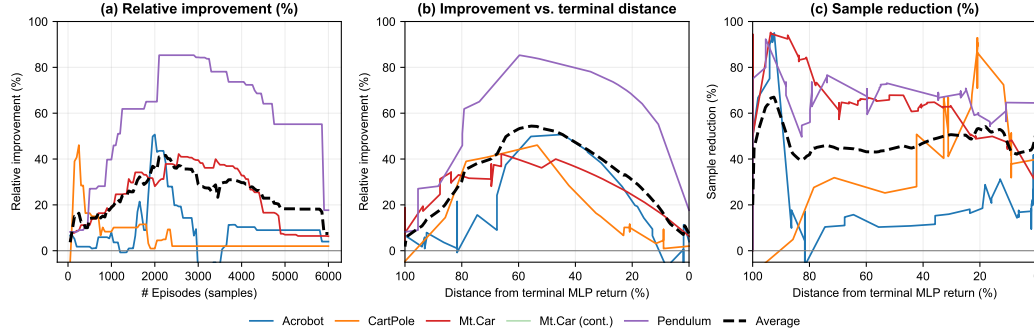


Figure 2: Experiment 2: Sample Efficiency in Gymnasium Benchmark.

References

- [1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [2] Bahare Kiumarsi, Kyriakos G Vamvoudakis, Hamidreza Modares, and Frank L Lewis. Optimal and autonomous control using reinforcement learning: A survey. *IEEE transactions on neural networks and learning systems*, 29(6):2042–2062, 2017.
- [3] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv Preprints*, 2017.
- [6] Chuheng Zhang, Yuanying Cai, Longbo Huang, and Jian Li. Exploration by maximizing renyi entropy for reward-free rl framework. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(12):10859–10867, 2021.
- [7] Jonas Eschmann. Reward function design in reinforcement learning. *Reinforcement learning algorithms: Analysis and Applications*, pages 25–33, 2021.
- [8] Peilang Li, Umer Siddique, and Yongcan Cao. From explainability to interpretability: Interpretable policies in reinforcement learning via model explanation. *arXiv Preprints*, 2025.
- [9] Silvia Ferrari and Robert F Stengel. Smooth function approximation using neural networks. *IEEE Transactions on Neural Networks*, 16(1):24–38, 2005.
- [10] Junzi Zhang, Jongho Kim, Brendan O’Donoghue, and Stephen Boyd. Sample efficient reinforcement learning with reinforce. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 10887–10895, 2021.
- [11] Fei Tao, Bin Xiao, Qinglin Qi, Jiangfeng Cheng, and Ping Ji. Digital twin modeling. *Journal of Manufacturing Systems*, 64:372–389, 2022.
- [12] Andrey N. Kolmogorov. On the representation of continuous functions of several variables by superpositions of continuous functions of fewer variables. *Doklady Akademii Nauk SSSR*, 108:179–182, 1956.
- [13] Vladimir I. Arnold. On functions of three variables. *Doklady Akademii Nauk SSSR*, 114:679–681, 1963.

- [14] Ziming Liu, Yifan Wang, Shashank Vaidya, Fabian Ruehle, James Halverson, Marin Soljagic, Thomas Y. Hou, and Max Tegmark. Kan: Kolmogorov-arnold networks. *arXiv Preprints*, 2024.
- [15] Shriyank Somvanshi, Syed Aaqib Javed, Md Monzurul Islam, Diwas Pandit, and Subasish Das. A survey on kolmogorov-arnold networks. *ACM Computing Surveys*, 58(2):1–35, 2025.
- [16] Siham Essahraoui, Ismail Lamaakal, Khalid El Makkaoui, Ibrahim Ouahbi, Mouncef Filali Bouami, and Yassine Maleh. Kolmogorov–arnold networks: Overview of architectures and use cases. In *International Conference on Circuits, Systems and Communication*, 2025.
- [17] Shairoz Sohail. On training of kolmogorov-arnold networks. *arXiv Preprints*, 2024.
- [18] Victor A. Kich, Jair A. Bottega, Raul Steinmetz, Ricardo B. Grando, Ayano Yoroazu, and Akihisa Ohya. Kolmogorov-arnold networks for online reinforcement learning. In *2024 24th International Conference on Control, Automation and Systems (ICCAS)*, pages 958–963. IEEE, 2024.
- [19] Ali Bayeh, Malek Mouhoub, and Samira Sadaoui. Kan-based on-policy ppo method for continuous reinforcement learning. In *Optimization and Learning: 8th International Conference, OLA 2025, Revised Selected Papers*, pages 3–15. Springer Nature Switzerland, 2026.
- [20] Zikang Chen, Fei Gao, Ziya Yu, and Peng Li. Kan policy: Learning efficient and smooth robotic trajectories via kolmogorov-arnold networks. *IEEE Robotics and Automation Letters*, 10(11):11164–11171, 2025.
- [21] Kamal Singh, Sami Marouani, Ahmad Al Sheikh, Pham Tran Anh Quang, and Amaury Habrard. Interpretable reinforcement learning for load balancing using kolmogorov-arnold networks. In *2026 IEEE 23rd Consumer Communications and Networking Conference (CCNC)*. IEEE, 2026.
- [22] Michele Antonio D’Alto. *Kolmogorov-Arnold Networks for Explainable Reinforcement Learning: A Symbolic Approach to Function Approximation in Reinforcement Learning*. PhD thesis, KTH Royal Institute of Technology, 2025.
- [23] Amanda A. Howard, Bruno Jacob, and Panos Stinis. Multifidelity kolmogorov-arnold networks. *arXiv Preprints*, 2024.
- [24] Enrique Luna Villagomez and Vladimir Mahalec. Comparative evaluation of kolmogorov-arnold autoencoders and orthogonal autoencoders for fault detection with varying training set sizes. *arXiv Preprints*, 2025.
- [25] Allen Wang and Gavin Tao. Quadkan: Kan-enhanced quadruped motion control via end-to-end reinforcement learning. *arXiv Preprints*, 2025.
- [26] Anastasis Kratsios, Bum Jun Kim, and Takashi Furuya. Approximation rates in besov norms and sample-complexity of kolmogorov-arnold networks with residual connections. *Neural Networks*, page 108797, 2026.
- [27] Farhad Pourkamali-Anaraki. Kolmogorov-arnold networks in low-data regimes: A comparative study with multilayer perceptrons. In *2025 International Conference on Machine Learning and Applications (ICMLA)*, pages 923–928. IEEE, 2025.
- [28] Chen Zeng, Jiahui Wang, Haoran Shen, and Qiao Wang. Kan versus mlp on irregular or noisy functions. *arXiv Preprints*, 2024.
- [29] Chenwei Shi and Xueyu Luan. Kan-dreamer: Benchmarking kolmogorov-arnold networks as function approximators in world models. *arXiv Preprints*, 2025.
- [30] Ziyao Li. Kolmogorov-arnold networks are radial basis function networks. *arXiv Preprints*, 2024.
- [31] Silviu-Marian Udrescu and Max Tegmark. Ai feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631, 2020.

- 259 [32] Silviu-Marian Udrescu, Andrew Tan, Jiahai Feng, Orisvaldo Neto, Tailin Wu, and Max Tegmark.
260 Ai feynman 2.0: Pareto-optimal symbolic regression exploiting graph modularity. In *Advances*
261 *in Neural Information Processing Systems*, volume 33, pages 4860–4871, 2020.
- 262 [33] Mark Towers, Ariel Kwiatkowski, John Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão,
263 Kallinteris Andreas, Markus Krimmel, Arjun Kg, Rodrigo Perez-Vicente, et al. Gymnasium: A
264 standard interface for reinforcement learning environments. *Advances in Neural Information*
265 *Processing Systems*, 38, 2026.
- 266 [34] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal
267 policy optimization algorithms. *arXiv Preprints*, 2017.
- 268 [35] Ziming Liu, Pingchuan Ma, Yixuan Wang, Wojciech Matusik, and Max Tegmark. Kan 2.0:
269 Kolmogorov-arnold networks meet science. *arXiv Preprints*, 2024.
- 270 [36] Hoang-Thang Ta. Bsrbf-kan: A combination of b-splines and radial basis functions in
271 kolmogorov-arnold networks. In *International Symposium on Information and Communication*
272 *Technology*, pages 3–15. Springer, 2024.
- 273 [37] Hoang-Thang Ta, Duy-Quy Thai, Abu Bakar Siddiquir Rahman, Grigori Sidorov, and Alexander
274 Gelbukh. Fc-kan: Function combinations in kolmogorov-arnold networks. *arXiv Preprints*,
275 2024.
- 276 [38] Hoang-Thang Ta, Duy-Quy Thai, Anh Tran, Grigori Sidorov, and Alexander Gelbukh. Prkan:
277 Parameter-reduced kolmogorov-arnold networks. *arXiv Preprints*, 2025.
- 278 [39] Hoang-Thang Ta and Anh Tran. Af-kan: Activation function-based kolmogorov-arnold networks
279 for efficient representation learning. *arXiv Preprints*, 2025.

280 A Experiment Setup Details

281 A.1 Feynman Benchmark Details

282 A.1.1 Dataset Construction

283 The Feynman benchmark consists of analytic equations with named input variables and prescribed
284 sampling ranges [31, 32]. Following Liu et al., we use the ‘Feynman_no_units’ subset and keep
285 all equations for which finite train and test samples can be generated [14] This includes univariate
286 equations, which are reported separately from multivariate equations in the aggregate analysis. We
287 use the unit-free subset, because univariate equations provide a sanity check for spline-based edge
288 learning, while multivariate equations test whether KANs retain any sample-efficiency advantage
289 when the model must learn interactions between variables. For each equation $f_j : \mathbb{R}^{d_j} \rightarrow \mathbb{R}$, we
290 generate supervised examples by sampling $x \sim \mathcal{U}(\mathcal{D}_j)$ and evaluating $y = f_j(x)$. The domain $\mathcal{D}_j$
291 is the Cartesian product of the variable intervals specified by the benchmark. Training and test sets
292 are sampled independently for every random seed. If an equation is undefined at a sampled point or
293 returns a non-finite value, we reject the point and resample until the desired split size is reached. This
294 rejection step is applied before normalization and is identical for all models.

295 A.1.2 Sample-Efficiency Protocol

296 For each equation, we fix $N_{\text{test}} = 1000$. We vary the training size over

$$N_{\text{train}} \in \{10, 20, 50, 100, 200, 500, 1000\}.$$

297 For each N_{train} , equation, and model family, we repeat the experiment over S random seeds. Each
298 seed determines the training draw, the test draw, and the model initialization. Inputs are standardized
299 using the mean and standard deviation of the training set. Targets may be standardized during
300 optimization, but all reported metrics are computed after transforming predictions back to the original
301 target scale. No information from the test set is used for model selection or normalization.

302 A.1.3 Model Comparisons

303 The main comparison includes KAN and MLP regressors trained with the same optimizer, loss,
304 maximum number of update steps, batch size, and random seeds. For KANs, we use spline-based
305 edge functions as in the original KAN implementation, with spline order and grid size fixed across
306 equations unless otherwise stated. For MLPs, we use smooth activations such as SiLU to avoid giving
307 KANs an artificial advantage over nonsmooth ReLU baselines on smooth physics equations. We
308 report trainable parameter counts and wall-clock training time for every trained model. The primary
309 setting uses a fixed architecture recipe for each model family. A secondary parameter-matched setting
310 chooses MLP widths so that the number of trainable parameters is as close as possible to the KAN
311 parameter count. When exact matching is impossible, we report the parameter ratio alongside the
312 result. This protocol avoids treating parameter count as interchangeable across architectures while
313 still checking whether apparent sample-efficiency gains are explained by model size.

314 A.1.4 KAN Implementations

315 To validate KAN baselines, we use the public All-KAN implementation repository as the implementa-
316 tion source for several KAN variants. The implementation collection is organized around the original
317 KAN model and the KAN 2.0 scientific-discovery framework [14, 35]. Our experiment wrapper
318 currently exposes EfficientKAN for B-spline KANs, FastKAN for Gaussian RBF edge functions,
319 and BSRBF-KAN for hybrid B-spline/RBF edge functions [30, 36]. The All-KAN repository also
320 includes recent variants such as FC-KAN, PRKAN, and AF-KAN, which provide implementation
321 references for future ablations [37, 38, 39]. All implementation choices are fixed across sample
322 budgets within an experiment, and each reported result specifies the backend used to instantiate the
323 KAN model.

324 A.1.5 Metrics

325 Let $\{(x_i, y_i)\}_{i=1}^{N_{\text{test}}}$ be the test set for one equation and let $\hat{y}_i$ be the model prediction. We define the
 326 test-set mean and standard deviation as

$$\bar{y} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} y_i, \quad s_y = \sqrt{\frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} (y_i - \bar{y})^2}.$$

327 The root mean squared error is

$$\text{RMSE} = \sqrt{\frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} (\hat{y}_i - y_i)^2}.$$

328 The normalized root mean squared error is

$$\text{NRMSE} = \frac{\text{RMSE}}{s_y + \epsilon},$$

329 where $\epsilon = 10^{-12}$ prevents numerical instability for nearly constant targets. The mean absolute error
 330 is

$$\text{MAE} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} |\hat{y}_i - y_i|.$$

331 The relative ℓ_2 error is

$$\text{Rel-}\ell_2 = \frac{\sqrt{\sum_{i=1}^{N_{\text{test}}} (\hat{y}_i - y_i)^2}}{\sqrt{\sum_{i=1}^{N_{\text{test}}} y_i^2 + \epsilon}}.$$

332 The coefficient of determination is

$$R^2 = 1 - \frac{\sum_{i=1}^{N_{\text{test}}} (\hat{y}_i - y_i)^2}{\sum_{i=1}^{N_{\text{test}}} (y_i - \bar{y})^2 + \epsilon}.$$

333 RMSE and MAE preserve the physical scale of each equation. NRMSE and relative ℓ_2 error allow
 334 comparisons across equations with different output magnitudes. R^2 reports explained variance and is
 335 easier to interpret when communicating whether a model has learned the dominant structure of the
 336 function.

337 A.1.6 Aggregate Reporting

338 For each equation and training size, we report the mean and standard error over seeds. Across
 339 equations, we report the median and interquartile range rather than only the mean, since a small
 340 number of high-scale or ill-conditioned equations can dominate aggregate errors. We report these
 341 aggregate statistics for all equations, for univariate equations only, and for multivariate equations
 342 only. The full sample-efficiency curve plots NRMSE against N_{train} on a logarithmic x-axis. As
 343 scalar summaries, we report the area under the error-sample curve and the sample threshold N_τ . The
 344 area under the error-sample curve is computed by trapezoidal integration of NRMSE over $\log N_{\text{train}}$,
 345 where lower values indicate better sample efficiency. The threshold N_τ is the smallest training-set
 346 size for which the median NRMSE falls below a chosen target τ . If no method reaches τ , we report
 347 $N_\tau > 1000$.

348 A.2 RL Gymnasium Benchmark Details

349 A.2.1 Benchmark and Agents

350 We use the Gymnasium Classic Control environments CartPole-v1, Acrobot-v1, MountainCar-v0,
 351 MountainCarContinuous-v0, and Pendulum-v1. The observations are low-dimensional state vectors,
 352 which matches the intended use of KAN edge functions for smooth physical relationships. The runner
 353 supports both discrete and continuous action spaces, using categorical policies for discrete actions
 354 and diagonal Gaussian policies with tanh squashing for continuous actions. For continuous actions,
 355 the trainable log standard deviation is initialised to 0.

356 **A.2.2 Model Sweep**

357 The main comparison uses full actor-critic agents, so the actor and critic are both KANs for KAN runs
358 and both MLPs for MLP runs. The KAN sweep includes B-spline KANs and Gaussian RBF KANs
359 with hidden dimensions [2], [4], [8], [4, 4], [8, 8], and [16, 16]. The MLP sweep includes ReLU and
360 sigmoid MLPs with hidden dimensions [16], [32], [64], [32, 32], and [64, 64]. The KAN settings use
361 grid size 5, spline order 3, grid range $[-1.5, 1.5]$, and SiLU as the base activation. The runner records
362 actor, critic, and total trainable parameter counts for every checkpoint. An actor-only diagnostic
363 mode exists in the code, but it is disabled for the main comparison.

364 **A.2.3 PPO Training and Evaluation**

365 Each training job collects complete episodes, computes generalised advantage estimates with $\gamma =$
366 0.99 and $\lambda = 0.95$, and applies clipped PPO updates with clip parameter 0.20 . The optimiser is
367 Adam with learning rate 3×10^{-4} , no weight decay, entropy coefficient 0.01 , value coefficient 0.50 ,
368 four PPO epochs per update, and gradient clipping at 0.50 . Updates are performed after batches of
369 five episodes in the full protocol. Training seeds are paired across model families, and evaluation uses
370 deterministic actions with five evaluation episodes per checkpoint. Discrete policies are evaluated
371 with the action of maximum logit, while continuous policies are evaluated with the mean action. The
372 result file used for the paper contains ten paired random seeds and learning curves up to 6000 training
373 episodes. The CSV logs training episodes, cumulative environment steps, recent training returns,
374 deterministic evaluation returns, episode lengths, PPO losses, runtime, and parameter counts.

375 **A.3 RL Gymnasium Noise Benchmark Details**

376 **A.3.1 Reward Perturbation**

377 The noisy benchmark keeps the same Gymnasium environments, PPO update, model sweep, and
378 deterministic clean evaluation protocol as the clean Gymnasium benchmark. During training only,
379 each clean reward r_t is replaced by $r_t^{\text{noisy}} = r_t + \alpha \epsilon_t$, where $\epsilon_t \sim \mathcal{N}(0, 1)$. The main sweep uses
380 $\alpha \in \{0, 0.25, 0.5, 1, 2, 4, 7, 10\}$ in raw reward units. The runner also supports an environment-scaled
381 mode, but the protocol file sets the scale mode to unit. Clean and noisy episode returns are logged
382 separately, so the plotted score remains the clean evaluation return. Because noise only enters the
383 rewards used for advantage estimation, the experiment tests robustness to corrupted training feedback
384 rather than robustness to stochastic evaluation.

385 **A.3.2 Noisy-Reward Logging**

386 The noisy protocol uses ten paired random seeds and runs up to 6000 training episodes per model
387 and noise setting. Each checkpoint stores the noise identifier, noise standard deviation, recent clean
388 return, recent noisy return, deterministic clean evaluation return, PPO losses, parameter counts, and
389 runtime. Completed jobs are detected at the full-job level, and partial jobs are removed and restarted
390 to avoid mixing trajectories.

391 **B Detailed Results**

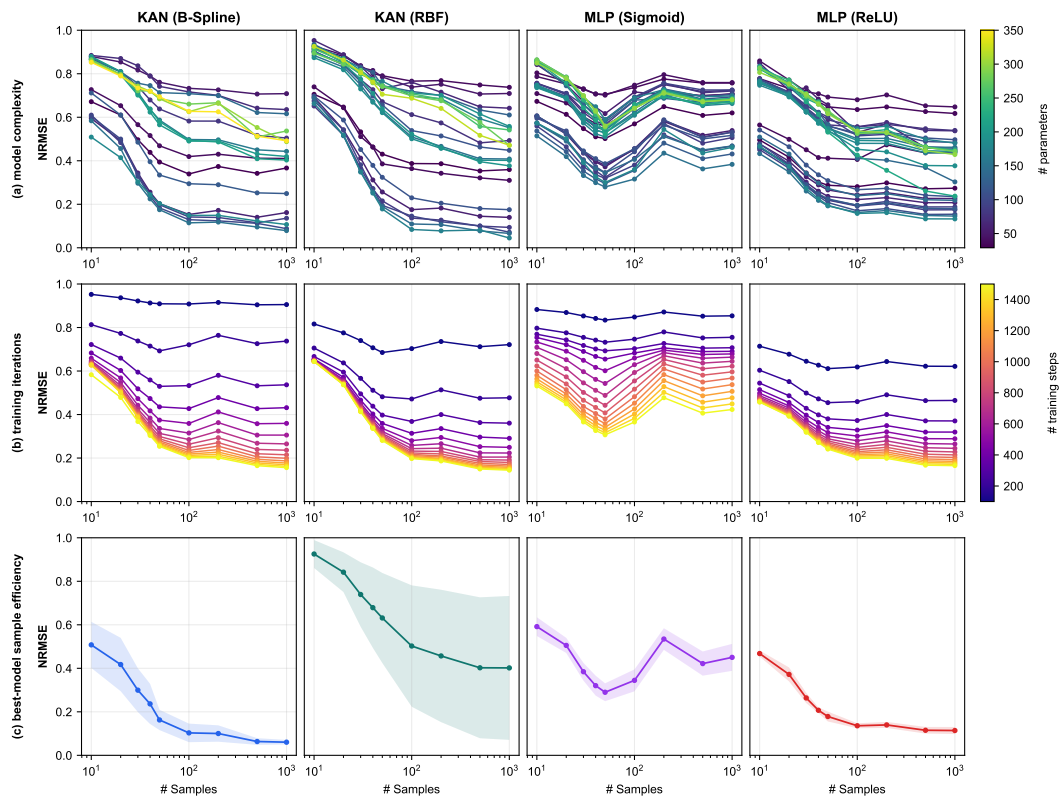


Figure 3: Experiment 1: Feynman Benchmark Experiment Design.

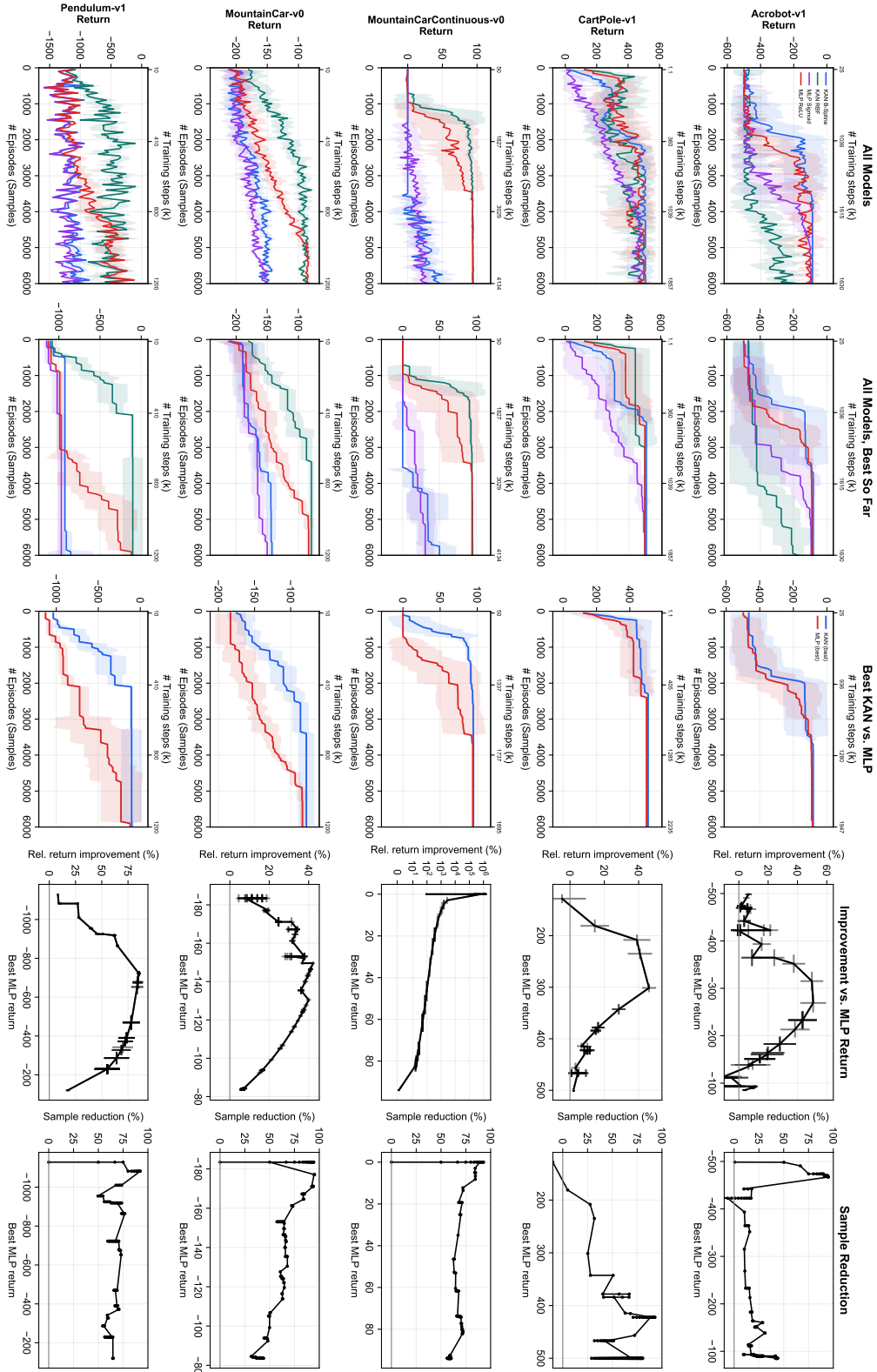


Figure 4: Experiment 2: DeepRL Gymnasium Experiment Design.

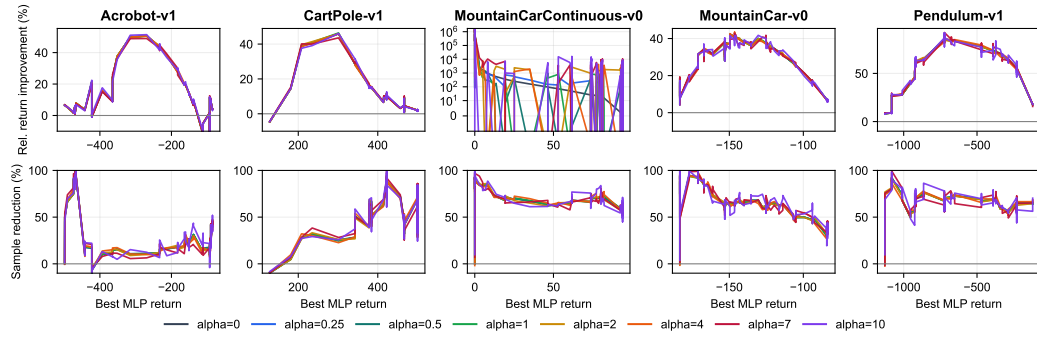


Figure 5: Experiment 3: Sample Efficiency in Noised Gymnasium Benchmark.